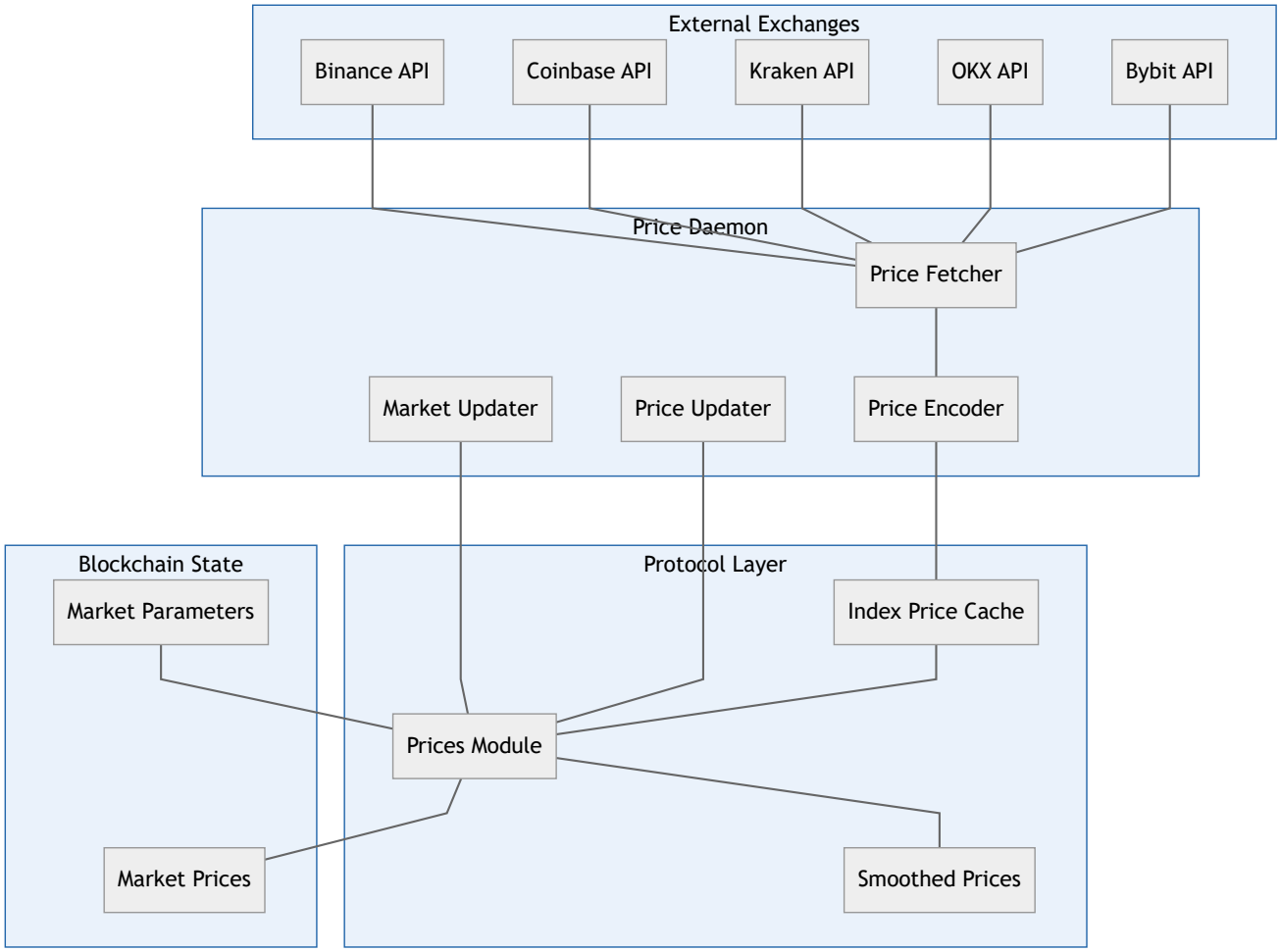
dYdX v4 Protocol Layer - 가격 피드 시스템 구현 분석

개요

dYdX v4의 가격 피드 시스템은 외부 거래소에서 실시간 가격 데이터를 수집하고, 이를 검증 및 집계하여 온체인 오라클 가격으로 제공하는 핵심 인프라입니다. 이 시스템은 Prices 모듈과 Price Daemon의 협력을 통해 구현되며, 고가용성과 정확성을 보장하기 위한 다양한 메커니즘을 포함합니다.

시스템 아키텍처

전체 구조



주요 컴포넌트

1. Prices 모듈 (Protocol Layer)

위치: `protocol/x/prices/`

역할: 온체인 가격 데이터 관리 및 오라클 기능

핵심 구조:

```
type Keeper struct {
```

```
cdc codec.BinaryCodec

storeKey storetypes.StoreKey

// 외부 거래소 가격 캐시

indexPriceCache *pricefeedtypes.MarketToExchangePrices

// 스무딩된 가격 데이터

marketToSmoothedPrices types.MarketToSmoothedPrices

// 시간 제공자

timeProvider libtime.TimeProvider

// 인덱서 이벤트 관리자

indexerEventManager indexer_manager.IndexerEventManager

// 시장 생성 시간 추적

marketToCreatedAt map[uint32]time.Time

// 권한 관리

authorities map[string]struct{}

}
```

2. Price Daemon

위치: `protocol/daemons/pricefeed/`

역할: 외부 거래소에서 가격 데이터 수집 및 전송

주요 서브태스크:

- **Price Fetcher**: 거래소 API에서 가격 수집
- **Price Encoder**: 수집된 가격 데이터 인코딩
- **Market Updater**: 시장 설정 업데이트

- **Price Updater**: 가격 데이터 전송

가격 데이터 수집 프로세스

1. 외부 거래소 연동

지원 거래소

```
// 주요 지원 거래소들
const (
EXCHANGE_ID_BINANCE = "binance"
EXCHANGE_ID_BINANCE_US = "binance_us"
EXCHANGE_ID_BITFINEX = "bitfinex"
EXCHANGE_ID_BITSTAMP = "bitstamp"
EXCHANGE_ID_BYBIT = "bybit"
EXCHANGE_ID_COINBASE_PRO = "coinbase_pro"
EXCHANGE_ID_CRYPTO_COM = "crypto_com"
EXCHANGE_ID_GATE = "gate"
EXCHANGE_ID_HUOBI = "huobi"
EXCHANGE_ID_KRAKEN = "kraken"
EXCHANGE_ID_KUCOIN = "kucoin"
EXCHANGE_ID_MEXC = "mexc"
EXCHANGE_ID_OKX = "okx"
)
```

거래소별 설정

```
type ExchangeQueryConfig struct {
ExchangeId string
IntervalMs uint32 // 쿼리 간격 (밀리초)
TimeoutMs uint32 // 타임아웃 (밀리초)
MaxQueries uint32 // 최대 동시 쿼리 수
}

type ExchangeQueryDetails struct {
Exchange ExchangeId
Url string
PriceFunction ExchangeQueryHandler // 가격 파싱 함수
IsMultiMarket bool // 다중 시장 지원 여부
}
```

2. Price Fetcher 구현

가격 수집 로직

```
func (p *PriceFetcher) fetchPricesForMarkets(
ctx context.Context,
marketIds []types.MarketId,
) []*PriceFetcherSubtaskResponse {
```

```go
var responses []*PriceFetcherSubtaskResponse

if p.exchangeDetails.IsMultiMarket {

// 다중 시장 지원 거래소: 한 번의 API 호출로 모든 시장 가격 수집

response := p.fetchMultiMarketPrices(ctx, marketIds)

responses = append(responses, response)

} else {

// 단일 시장 거래소: 각 시장별로 개별 API 호출

for _, marketId := range marketIds {

response := p.fetchSingleMarketPrice(ctx, marketId)

responses = append(responses, response)

}

}

return responses

}



func (p *PriceFetcher) fetchSingleMarketPrice(

ctx context.Context,

marketId types.MarketId,

) *PriceFetcherSubtaskResponse {

// 1. 시장 설정 가져오기

marketConfig := p.getMarketConfig(marketId)

if marketConfig == nil {

return &PriceFetcherSubtaskResponse{
```

```
Err: fmt.Errorf("market config not found for market %v", marketId),

}

}

// 2. API 요청 실행

requestUrl := p.buildRequestUrl(marketConfig)

response, err := p.queryHandler.Query(ctx, requestUrl, p.exchangeQueryConfig.Tim

if err != nil {

return &PriceFetcherSubtaskResponse{Err: err}

}

// 3. 응답 파싱

price, err := p.parsePrice(response, marketConfig)

if err != nil {

return &PriceFetcherSubtaskResponse{Err: err}

}

// 4. 가격 검증

if err := p.validatePrice(price, marketConfig); err != nil {

return &PriceFetcherSubtaskResponse{Err: err}

}

return &PriceFetcherSubtaskResponse{

Price: &types.MarketPriceTimestamp{

MarketId: marketId,

Price: price,

Timestamp: p.timeProvider.Now(),
```

```
},

}

}
```

에러 처리 및 재시도

```
func (p *PriceFetcher) executeWithRetry(

ctx context.Context,

operation func() (*PriceFetcherSubtaskResponse, error),

maxRetries int,

) *PriceFetcherSubtaskResponse {

var lastErr error

for attempt := 0; attempt <= maxRetries; attempt++ {

if attempt > 0 {

// 지수 백오프로 재시도 간격 증가

backoffDuration := time.Duration(attempt*attempt) * time.Second

time.Sleep(backoffDuration)

}

response, err := operation()

if err == nil {

return response

}

lastErr = err

p.logger.Warn("Price fetch attempt failed",
```

```
"attempt", attempt+1,
"maxRetries", maxRetries+1,
"error", err)
}
return &PriceFetcherSubtaskResponse{Err: lastErr}
}
```

3. Price Encoder

가격 데이터 인코딩

```
func (pe *PriceEncoder) encodePrices(
responses []*PriceFetcherSubtaskResponse,
) {
var validPrices []*types.MarketPriceTimestamp
for _, response := range responses {
if response.Err != nil {
pe.logger.Error("Price fetch error", "error", response.Err)
pe.recordError(response.Err)
continue
}
if response.Price == nil {
pe.logger.Warn("Received nil price")
continue
```

```
}
// 가격 유효성 검증
if pe.isValidPrice(response.Price) {
validPrices = append(validPrices, response.Price)
}
}
// 유효한 가격들을 캐시에 저장
pe.updatePriceCache(validPrices)
}


func (pe *PriceEncoder) isValidPrice(price *types.MarketPriceTimestamp) bool {
// 1. 가격이 양수인지 확인
if price.Price <= 0 {
return false
}
// 2. 타임스탬프가 너무 오래되지 않았는지 확인
maxAge := 30 * time.Second
if time.Since(price.Timestamp) > maxAge {
return false
}
// 3. 시장별 가격 범위 검증
if !pe.isPriceInValidRange(price.MarketId, price.Price) {
return false
```

```
}
return true
}
```

가격 검증 및 이상치 탐지

1. 다중 거래소 가격 집계

중앙값 계산

```
func (mte *MarketToExchangePrices) GetValidMedianPrices(
marketParams []types.MarketParam,
readTime time.Time,
) map[uint32]uint64 {
cutoffTime := readTime.Add(-mte.maxPriceAge)
marketIdToMedianPrice := make(map[uint32]uint64)
mte.Lock()
defer mte.Unlock()
for _, marketParam := range marketParams {
marketId := marketParam.Id
exchangeToPrice, ok := mte.marketToExchangePrices[marketId]
if !ok {
continue
}
```

```
// 유효한 가격들만 필터링 (시간 기준)
validPrices := exchangeToPrice.GetValidPrices(cutoffTime)
// 최소 거래소 수 확인
if len(validPrices) < marketParam.MinExchanges {
continue
}
// 중앙값 계산
median, err := lib.Median(validPrices)
if err != nil {
continue
}
marketIdToMedianPrice[marketId] = median
}
return marketIdToMedianPrice
}
```

이상치 탐지 알고리즘

```
func (k Keeper) detectPriceOutliers(
marketId uint32,
newPrice uint64,
historicalPrices []uint64,
) bool {
```

```
    if len(historicalPrices) < 10 {
    return false // 충분한 데이터가 없으면 이상치로 판단하지 않음
    }
    // 1. Z-Score 기반 이상치 탐지
    mean := calculateMean(historicalPrices)
    stdDev := calculateStandardDeviation(historicalPrices, mean)
    zScore := math.Abs(float64(newPrice-mean)) / stdDev
    if zScore > 3.0 { // 3-시그마 규칙
    return true
    }
    // 2. IQR (Interquartile Range) 기반 이상치 탐지
    q1, q3 := calculateQuartiles(historicalPrices)
    iqr := q3 - q1
    lowerBound := q1 - 1.5*iqr
    upperBound := q3 + 1.5*iqr
    if float64(newPrice) < lowerBound || float64(newPrice) > upperBound {
    return true
    }
    // 3. 급격한 가격 변동 탐지
    recentPrice := historicalPrices[len(historicalPrices)-1]
    changeRate := math.Abs(float64(newPrice-recentPrice)) / float64(recentPrice)
    if changeRate > 0.1 { // 10% 이상 변동
    return true
```

```
}

return false

}
```

2. 가격 스무딩 (Price Smoothing)

지수 이동 평균 (EMA)

```
type SmoothedPrices struct {

prices []uint64

weights []float64

maxSamples int

}


func (sp *SmoothedPrices) UpdatePrice(newPrice uint64) uint64 {

sp.prices = append(sp.prices, newPrice)

// 최대 샘플 수 유지

if len(sp.prices) > sp.maxSamples {

sp.prices = sp.prices[1:]

}

// EMA 계산

alpha := 0.1 // 스무딩 팩터

if len(sp.prices) == 1 {

return newPrice
```

```
}

var ema float64 = float64(sp.prices[0])

for i := 1; i < len(sp.prices); i++ {

ema = alpha*float64(sp.prices[i]) + (1-alpha)*ema

}

return uint64(ema)

}
```

오라클 가격 업데이트 메커니즘

1. 가격 업데이트 프로세스

Price Updater 구현

```
func (pu *PriceUpdater) updatePrices(

ctx context.Context,

exchangeToMarketPrices *types.ExchangeToMarketPrices,

pricefeedClient api.PriceFeedServiceClient,

) error {

// 1. 모든 시장의 유효한 중앙값 가격 가져오기

marketParams := pu.getMarketParams(ctx)

validPrices := exchangeToMarketPrices.GetValidMedianPrices(

marketParams,

pu.timeProvider.Now(),
```

```
)

// 2. 가격 업데이트 메시지 생성

var updates []*api.MarketPriceUpdate

for marketId, price := range validPrices {

updates = append(updates, &api.MarketPriceUpdate{

MarketId: marketId,

Price: price,

})

}

if len(updates) == 0 {

return nil // 업데이트할 가격이 없음

}

// 3. gRPC를 통해 가격 업데이트 전송

request := &api.UpdateMarketPricesRequest{

MarketPriceUpdates: updates,

}

_, err := pricefeedClient.UpdateMarketPrices(ctx, request)

if err != nil {

return fmt.Errorf("failed to update market prices: %w", err)

}

// 4. 메트릭 업데이트

pu.updateMetrics(updates)

return nil
```

```
}
```

2. 온체인 가격 저장

Market Price 업데이트

```
func (k Keeper) UpdateMarketPrices(
ctx sdk.Context,
updates []*types.MsgUpdateMarketPrices_MarketPrice,
) error {
marketPriceStore := k.getMarketPriceStore(ctx)
updatedMarketPrices := make([]types.MarketPrice, 0, len(updates))
for _, update := range updates {
// 기존 시장 가격 가져오기
marketPrice, err := k.GetMarketPrice(ctx, update.MarketId)
if err != nil {
return err
}
// 가격 변동률 계산 및 메트릭 기록
if diffRate, err := lib.ChangeRateUint64(marketPrice.Price, update.Price); err =
telemetry.SetGaugeWithLabels(
[]string{types.ModuleName, metrics.PriceChangeRate},
diffRate,
[]gometrics.Label{
```

```
pricefeedmetrics.GetLabelForMarketId(marketPrice.Id),

},

)

}

// 가격 업데이트

marketPrice.Price = update.Price

updatedMarketPrices = append(updatedMarketPrices, marketPrice)

// 현재 가격 메트릭 기록

updatedPrice, _ := lib.BigMulPow10(

new(big.Int).SetUint64(update.Price),

marketPrice.Exponent,

).Float32()

telemetry.SetGaugeWithLabels(

[]string{types.ModuleName, metrics.CurrentMarketPrices},

updatedPrice,

[]gometrics.Label{

pricefeedmetrics.GetLabelForMarketId(marketPrice.Id),

},

)

}

// 원자적으로 모든 가격 업데이트 적용

for _, marketPrice := range updatedMarketPrices {

b := k.cdc.MustMarshal(&marketPrice)
```

```
marketPriceStore.Set(lib.Uint32ToKey(marketPrice.Id), b)

// 마지막 업데이트 블록 기록

telemetry.SetGaugeWithLabels(

[]string{types.ModuleName, metrics.LastPriceUpdateForMarketBlock},

float32(ctx.BlockHeight()),

[]gometrics.Label{

pricefeedmetrics.GetLabelForMarketId(marketPrice.Id),

},

)

}

// 인덱서 이벤트 생성

priceUpdateEvents := GenerateMarketPriceUpdateIndexerEvents(updatedMarketPrices)

for _, event := range priceUpdateEvents {

k.GetIndexerEventManager().AddTxnEvent(

ctx,

indexerevents.SubtypeMarket,

indexerevents.MarketEventVersion,

indexer_manager.GetBytes(event),

)

}

return nil

}
```

고가용성 및 장애 복구

1. 거래소 장애 대응

자동 페일오버

```
type ExchangeHealthMonitor struct {
exchangeStatus map[types.ExchangeId]*ExchangeStatus
mutex sync.RWMutex
}

type ExchangeStatus struct {
IsHealthy bool
LastSuccessTime time.Time
ConsecutiveErrors int
ErrorRate float64
}

func (ehm *ExchangeHealthMonitor) checkExchangeHealth(
exchangeId types.ExchangeId,
err error,
) {
ehm.mutex.Lock()
defer ehm.mutex.Unlock()
```

```
status, exists := ehm.exchangeStatus[exchangeId]

if !exists {

status = &ExchangeStatus{IsHealthy: true}

ehm.exchangeStatus[exchangeId] = status

}

if err != nil {

status.ConsecutiveErrors++

status.ErrorRate = calculateErrorRate(exchangeId)

// 연속 에러가 임계값을 초과하면 비정상으로 표시

if status.ConsecutiveErrors >= 5 || status.ErrorRate > 0.5 {

status.IsHealthy = false

}

} else {

status.LastSuccessTime = time.Now()

status.ConsecutiveErrors = 0

status.IsHealthy = true

}

}


func (ehm *ExchangeHealthMonitor) getHealthyExchanges() []types.ExchangeId {

ehm.mutex.RLock()

defer ehm.mutex.RUnlock()

var healthyExchanges []types.ExchangeId
```

```
for exchangeId, status := range ehm.exchangeStatus {
if status.IsHealthy {
healthyExchanges = append(healthyExchanges, exchangeId)
}
}
return healthyExchanges
}
```

2. 가격 데이터 백업 및 복구

가격 히스토리 관리

```
type PriceHistoryManager struct {
priceHistory map[uint32]*CircularBuffer // marketId -> price history
maxHistory int
}

func (phm *PriceHistoryManager) AddPrice(marketId uint32, price uint64, timestam
if phm.priceHistory[marketId] == nil {
phm.priceHistory[marketId] = NewCircularBuffer(phm.maxHistory)
}
pricePoint := &PricePoint{
Price: price,
Timestamp: timestamp,
```

```go
}

phm.priceHistory[marketId].Add(pricePoint)

}



func (phm *PriceHistoryManager) GetLastValidPrice(marketId uint32) (*PricePoint,

history := phm.priceHistory[marketId]

if history == nil || history.IsEmpty() {

return nil, fmt.Errorf("no price history for market %d", marketId)

}

// 가장 최근의 유효한 가격 반환

for i := history.Size() - 1; i >= 0; i-- {

pricePoint := history.Get(i).(*PricePoint)

if time.Since(pricePoint.Timestamp) < 5*time.Minute {

return pricePoint, nil

}

}

return nil, fmt.Errorf("no recent valid price for market %d", marketId)

}
```

성능 최적화

1. 동시성 처리

병렬 가격 수집

```go
func (pf *PriceFetcher) fetchPricesParallel(
ctx context.Context,
marketIds []types.MarketId,
) []*PriceFetcherSubtaskResponse {
numWorkers := min(len(marketIds), pf.maxConcurrentQueries)
marketChan := make(chan types.MarketId, len(marketIds))
resultChan := make(chan *PriceFetcherSubtaskResponse, len(marketIds))
// 작업 큐에 시장 ID 추가
for _, marketId := range marketIds {
marketChan <- marketId
}
close(marketChan)
// 워커 고루틴 시작
var wg sync.WaitGroup
for i := 0; i < numWorkers; i++ {
wg.Add(1)
go func() {
defer wg.Done()
for marketId := range marketChan {
response := pf.fetchSingleMarketPrice(ctx, marketId)
resultChan <- response
}
```

```
}()

}

// 결과 수집

go func() {

wg.Wait()

close(resultChan)

}()

var responses []*PriceFetcherSubtaskResponse

for response := range resultChan {

responses = append(responses, response)

}

return responses

}
```

2. 캐싱 최적화

메모리 효율적인 가격 캐시

```
type OptimizedPriceCache struct {

prices map[uint32]*PriceEntry

lruList *list.List

maxSize int

mutex sync.RWMutex

}
```

```go
type PriceEntry struct {
MarketId uint32
Price uint64
Timestamp time.Time
Element *list.Element
}


func (opc *OptimizedPriceCache) Set(marketId uint32, price uint64, timestamp tim
opc.mutex.Lock()
defer opc.mutex.Unlock()
if entry, exists := opc.prices[marketId]; exists {
// 기존 엔트리 업데이트
entry.Price = price
entry.Timestamp = timestamp
opc.lruList.MoveToFront(entry.Element)
} else {
// 새 엔트리 추가
entry := &PriceEntry{
MarketId: marketId,
Price: price,
Timestamp: timestamp,
}
```

```
entry.Element = opc.lruList.PushFront(entry)

opc.prices[marketId] = entry

// 캐시 크기 제한

if len(opc.prices) > opc.maxSize {

opc.evictLRU()

}

}

}



func (opc *OptimizedPriceCache) evictLRU() {

if opc.lruList.Len() == 0 {

return

}

oldest := opc.lruList.Back()

if oldest != nil {

entry := oldest.Value.(*PriceEntry)

delete(opc.prices, entry.MarketId)

opc.lruList.Remove(oldest)

}

}
```

모니터링 및 알림

1. 메트릭 수집

가격 피드 메트릭

```
func (k Keeper) recordPriceFeedMetrics(ctx sdk.Context) {
allMarketPrices := k.GetAllMarketPrices(ctx)
for _, marketPrice := range allMarketPrices {
// 현재 가격 기록
telemetry.SetGaugeWithLabels(
[]string{types.ModuleName, metrics.CurrentMarketPrices},
float32(marketPrice.Price),
[]gometrics.Label{
pricefeedmetrics.GetLabelForMarketId(marketPrice.Id),
},
)
// 마지막 업데이트 시간 기록
telemetry.SetGaugeWithLabels(
[]string{types.ModuleName, metrics.LastPriceUpdateTime},
float32(time.Now().Unix()),
[]gometrics.Label{
pricefeedmetrics.GetLabelForMarketId(marketPrice.Id),
},
)
}
```

```
// 전체 시장 수 기록
telemetry.SetGauge(
[]string{types.ModuleName, metrics.TotalMarkets},
float32(len(allMarketPrices)),
)
}
```

2. 알림 시스템

가격 이상 탐지 알림

```
func (k Keeper) checkPriceAnomalies(ctx sdk.Context) {
allMarketPrices := k.GetAllMarketPrices(ctx)
for _, marketPrice := range allMarketPrices {
// 가격 변동률 확인
if k.isPriceAnomalous(marketPrice) {
k.sendPriceAlert(ctx, marketPrice, "PRICE_ANOMALY")
}
// 가격 업데이트 지연 확인
if k.isPriceStale(marketPrice) {
k.sendPriceAlert(ctx, marketPrice, "STALE_PRICE")
}
// 거래소 연결 상태 확인
if k.isExchangeUnhealthy(marketPrice.Id) {
```

```
k.sendPriceAlert(ctx, marketPrice, "EXCHANGE_UNHEALTHY")

}

}

}


func (k Keeper) sendPriceAlert(

ctx sdk.Context,

marketPrice types.MarketPrice,

alertType string,

) {

alert := types.PriceAlert{

MarketId: marketPrice.Id,

Price: marketPrice.Price,

AlertType: alertType,

Timestamp: ctx.BlockTime(),

}

// 로그 기록

k.Logger(ctx).Error("Price alert triggered",

"marketId", alert.MarketId,

"price", alert.Price,

"alertType", alert.AlertType,

)

// 메트릭 증가
```

```
telemetry.IncrCounterWithLabels(

[]string{types.ModuleName, metrics.PriceAlerts},

1,

[]gometrics.Label{

pricefeedmetrics.GetLabelForMarketId(alert.MarketId),

{Name: "alert_type", Value: alert.AlertType},

},

)

}
```

결론

dYdX v4의 가격 피드 시스템은 다중 거래소에서 실시간 가격 데이터를 안정적으로 수집하고, 이를 검증 및 집계하여 정확한 오라클 가격을 제공하는 정교한 시스템입니다. Price Daemon의 병렬 처리와 Prices 모듈의 온체인 검증을 통해 높은 가용성과 정확성을 보장하며, 이상치 탐지와 스무딩 알고리즘을 통해 시장 조작이나 일시적인 가격 급변동으로부터 시스템을 보호합니다. 이러한 견고한 가격 피드 인프라는 dYdX v4가 안전하고 신뢰할 수 있는 탈중앙화 영구 선물 거래소로 운영될 수 있는 핵심 기반을 제공합니다.